

(continued)

type *A* forms a monoid under the operations defined by the `Monoid[A]` instance. Less precisely, we might say that “type *A* is a monoid,” or even that “type *A* is *monoidal*.” In any case, the `Monoid[A]` instance is simply evidence of this fact.

This is much the same as saying that the page or screen you’re reading “forms a rectangle” or “is rectangular.” It’s less accurate to say that it “is a rectangle” (although that still makes sense), but to say that it “has a rectangle” would be strange.

Just what *is* a monoid, then? It’s simply a type *A* and an implementation of `Monoid[A]` that satisfies the laws. Stated tersely, *a monoid is a type together with a binary operation (op) over that type, satisfying associativity and having an identity element (zero).*

What does this buy us? Just like any abstraction, a monoid is useful to the extent that we can write useful generic code assuming only the capabilities provided by the abstraction. Can we write any interesting programs, knowing nothing about a type other than that it forms a monoid? Absolutely! Let’s look at some examples.

10.2 Folding lists with monoids

Monoids have an intimate connection with lists. If you look at the signatures of `foldLeft` and `foldRight` on `List`, you might notice something about the argument types:

```
def foldRight[B](z: B)(f: (A, B) => B): B
def foldLeft[B](z: B)(f: (B, A) => B): B
```

What happens when *A* and *B* are the same type?

```
def foldRight(z: A)(f: (A, A) => A): A
def foldLeft(z: A)(f: (A, A) => A): A
```

The components of a monoid fit these argument types like a glove. So if we had a list of `Strings`, we could simply pass the `op` and `zero` of the `stringMonoid` in order to reduce the list with the monoid and concatenate all the strings:

```
scala> val words = List("Hic", "Est", "Index")
words: List[String] = List(Hic, Est, Index)

scala> val s = words.foldRight(stringMonoid.zero)(stringMonoid.op)
s: String = "HicEstIndex"

scala> val t = words.foldLeft(stringMonoid.zero)(stringMonoid.op)
t: String = "HicEstIndex"
```

Note that it doesn’t matter if we choose `foldLeft` or `foldRight` when folding with a monoid;³ we should get the same result. This is precisely because the laws of associativity and identity hold. A left fold associates operations to the left, whereas a right fold associates to the right, with the identity element on the left and right respectively:

³ Given that both `foldLeft` and `foldRight` have tail-recursive implementations.